Universidade Franciscana
Somos universidade. Somos você.

Curso: Ciência da Computação

Aplicativos Móveis

aula 03

Listar aluno

Professor: André Flores dos Santos

CC BY NC SA

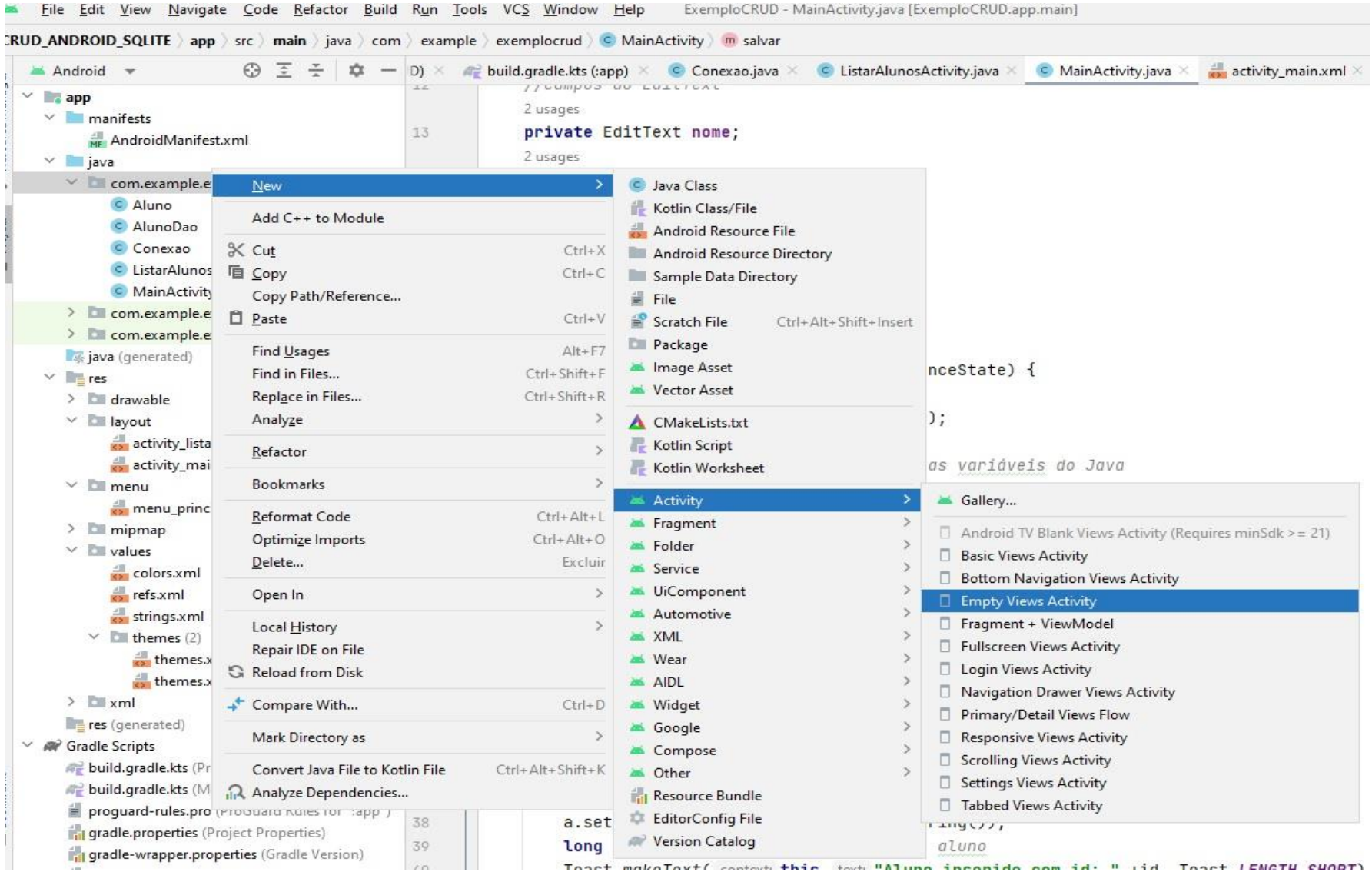
Santa Maria – RS 2026

Listar Aluno

- Continuação do exemplo da aula anterior CRUD Android
- Hoje vamos desenvolver o ‘Listar Aluno’

Listar Aluno

Criar uma nova activity



Clicar com botão direito em cima do pacote.

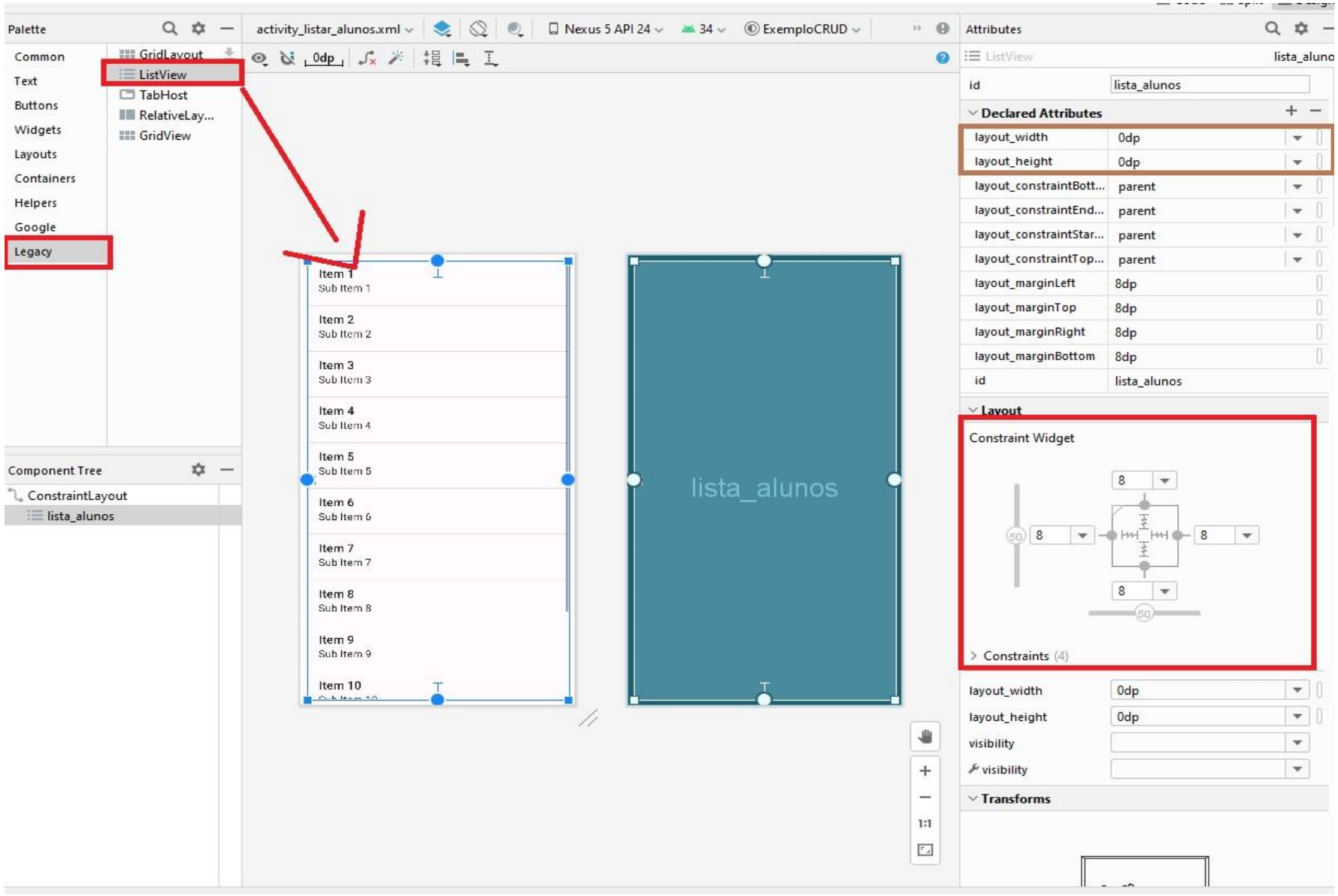
New>Activity>Empty Views Activity

Colocar o nome: ListarAlunosActivity ou o que preferir

Marcar o campo ‘criar arquivo de layout.xml’ ou ‘Generate a Layout File’, caso contrário terá que ir e criar ele (layout = a parte gráfica alocada a ele).

Listar Aluno

Inserir um ListView



Ir em Legacy>ListView e inserir o componente sobre a tela.

Definir os atributos layout_width e height com match constraint para ocupar a tela conforme foi definido.

Ancorar os pontos na tela como anteriormente, com margens = 8

Classe Aluno.java

```java
public class Aluno implements Serializable {
    private Integer id;
    private String nome;
    private String cpf;
    private String telefone;

    public Integer getId() {
        return id;
    }
    public void setId(Integer id) {
        this.id = id;
    }
    public String getNome() {
        return nome;
    }
    public void setNome(String nome) {
        this.nome = nome;
    }
    public String getCpf() {
        return cpf;
    }
    public void setCpf(String cpf) {
        this.cpf = cpf;
    }
    public String getTelefone() {
        return telefone;
    }
    public void setTelefone(String telefone) {
        this.telefone = telefone;
    }

    //facilitar a visualização de dados e passar o aluno como string
    //sobreescrever o método ToString
    //Quando o aluno for convertido para String irá mostrar somente o nome dele
    @Override
    public String toString(){

        return nome;
    }
}
```

Atualizar o nome do pacote do seu projeto e bibliotecas necessárias.

Classe ListarAlunosActivity.java

```java
package com.example.exemplocrud;

import androidx.appcompat.app.AppCompatActivity;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuInflater;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import java.util.ArrayList;
import java.util.List;

//PARTE 2
public class ListarAlunosActivity extends AppCompatActivity {

    private ListView listView;
    private AlunoDao dao;
    private List<Aluno> alunos;


    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_listar_alunos);

        //vincular variaveis com os campos do layout
        listView = findViewById(R.id.lista_alunos);
        dao = new AlunoDao(this);
        alunos = dao.obterTodos(); //todos alunos

        //ArrayAdapter já vem pronto no android para colocar essa lista de alunos na listview
        ArrayAdapter<Aluno> adaptador = new ArrayAdapter<Aluno>(this, android.R.layout.simple_list_item_1, alunos);
        //colocar na listView o adaptador
        listView.setAdapter(adaptador);
    }

}
```

Atualizar o nome do pacote do seu projeto e bibliotecas necessárias

Classe AlunoDao.java

Atualizar o projeto e bibliotecas necessárias

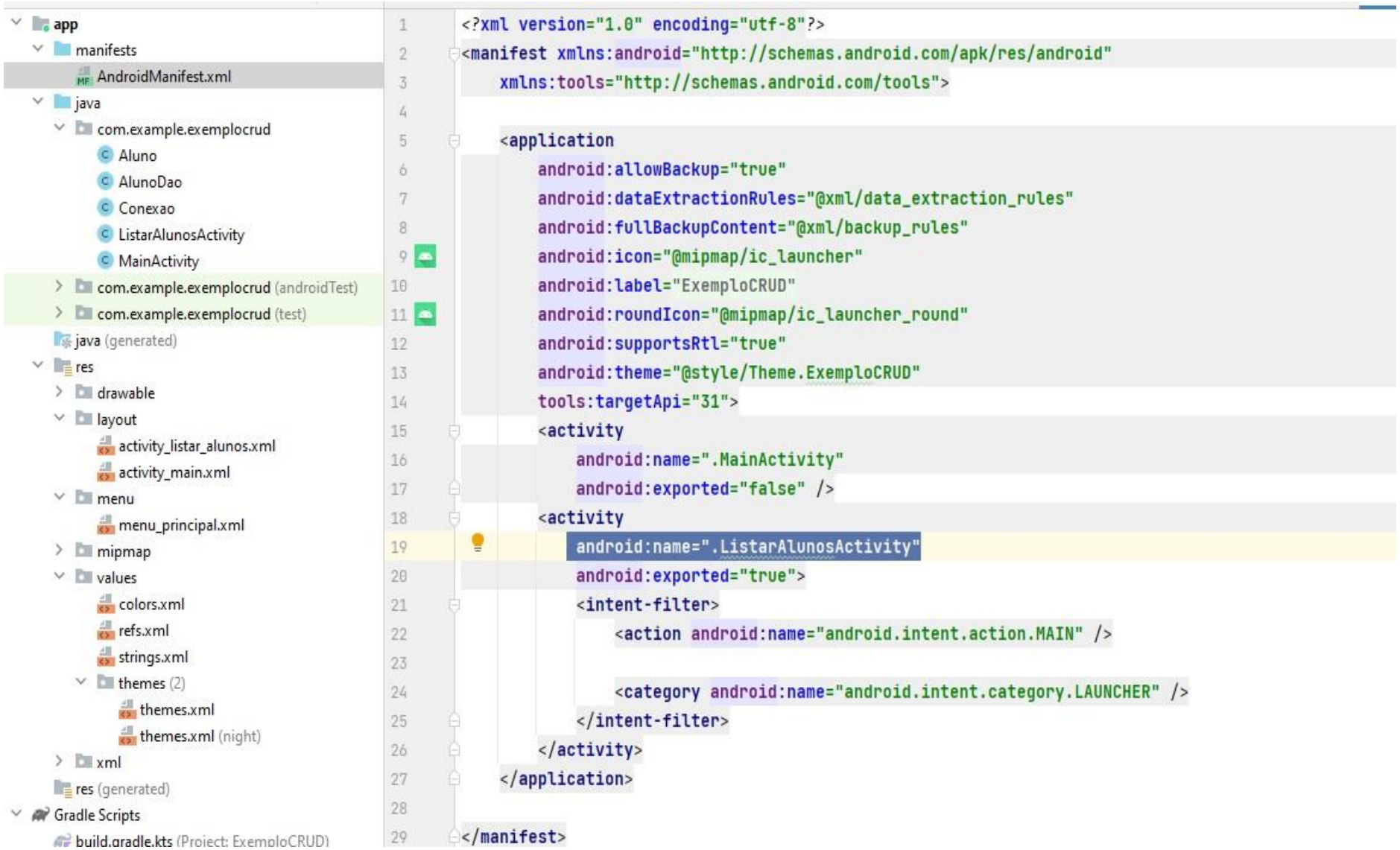
```
public class AlunoDao {
    private Conexao conexao;
    private SQLiteDatabase banco;

    //context é usado para a conexão
    public AlunoDao(Context context){
        conexao = new Conexao(context); //criei uma conexao
        banco = conexao.getWritableDatabase(); //iniciar um banco de dados para escrita
    }
    //método para inserir - PARTE I
    public long inserir(Aluno aluno){ // long porque retorna o id do aluno
        ContentValues values = new ContentValues(); //valores que irei inserir
        values.put("nome", aluno.getNome());
        values.put("cpf", aluno.getCpf());
        values.put("telefone", aluno.getTelefone());
        return banco.insert("aluno",null, values); //tabela aluno, não tera colunas vazias, valores values
    }
    //método para consultar PARTE II
    public List<Aluno> obterTodos(){
        List<Aluno> alunos = new ArrayList<>();
        //cursor aponta para as linhas retornadas
        Cursor cursor = banco.query("aluno", new String[]{"id", "nome", "cpf", "telefone"},
                null, null,null,null,null); //nome da tabela, nome das colunas, completa com null o método
                //que por padrão pede esse número de colunas obrigatórias
        while(cursor.moveToNext()){ //verifica se consegue mover para o próximo ponteiro ou linha
            Aluno a = new Aluno();
            a.setId(cursor.getInt(0)); // new String[]{"id", "nome", "cpf", "telefone"}, id é coluna '0'
            a.setNome(cursor.getString(1)); // new String[]{"id", "nome", "cpf", "telefone"}, nome é coluna '1'
            a.setCpf(cursor.getString(2)); // new String[]{"id", "nome", "cpf", "telefone"}, cpf é coluna '2'
            a.setTelefone(cursor.getString(3)); // new String[]{"id", "nome", "cpf", "telefone"}, telefone é coluna '3'
            alunos.add(a);
        }
        return alunos;
    }
}
```

Antes de rodar o projeto, alterar a activity principal

Agora nós queremos testar a tela do ListarAluno, porém ainda não tem um botão para nos levar até ela.

Vamos colocar ela como tela principal para quando abrir o App, como mudar isso?

Antes de rodar o projeto, alterar a activity principal



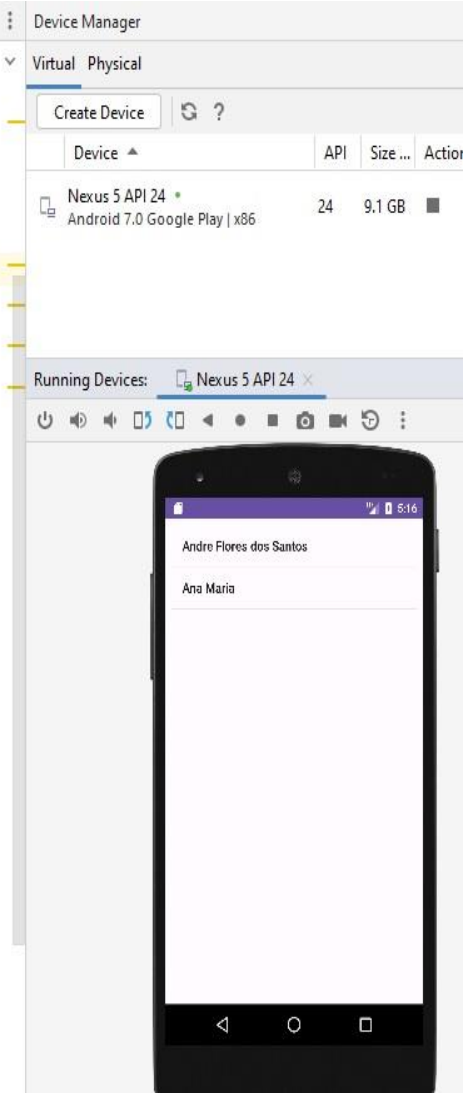
Detalhe: temos que mudar a activity principal para ‘ListarAlunosActivity’, porque se não quando rodarmos o projeto, a tela de cadastro irá aparecer lá, pois ela era a activity principal até o momento!

Clicar na raiz do projeto App>manifests> abrir AndroidManifest.xml

Devemos colocar ‘true’ na activity que queremos como principal e a tag <intente-filter> e seu conteúdo devem estar apenas na activity principal, na outra deve ser comentada ou apagada.

Resultado ListarAlunosActivity

```
oDao.java  Conexao.java  ListarAlunosActivity.java  MainActivity.java  activity_main.xml  Aluno.java  AndroidManifest.xml  activity_lista
private ListView listView;
2 usages
private AlunoDao dao;
3 usages
private List<Aluno> alunos;
1 usage
private List<Aluno> alunosFiltrados = new ArrayList<>();


@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_listar_alunos);


    //vincular variaveis com os campos do layout
    listView = findViewById(R.id.lista_alunos);
    dao = new AlunoDao( context: this);
    alunos = dao.obterTodos(); //todos alunos
    alunosFiltrados.addAll(alunos); //só os alunos que foram consultados

    //ArrayAdapter já vem pronto no android para colocar essa lista de alunos na listview
    ArrayAdapter<Aluno> adaptador = new ArrayAdapter<~>( context: this, android.R.layout.simple_list_item_1, alunos);
    //colocar na listView o adaptador
    listView.setAdapter(adaptador);
}


public boolean onCreateOptionsMenu(Menu menu){
    MenuInflater i = getMenuInflater();
    i.inflate(R.menu.menu_principal, menu);
    return true;
```



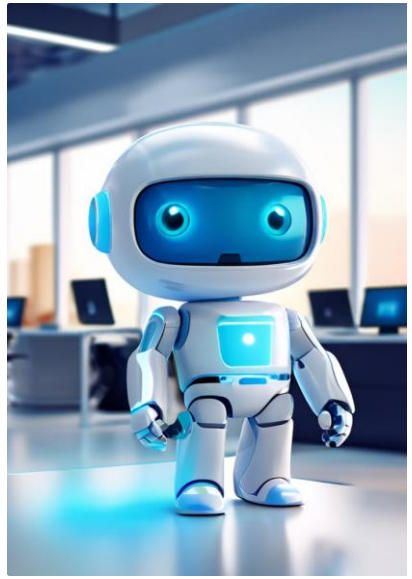
Após rodar o projeto, podemos visualizar todos os nomes que **foram cadastrados na etapa anterior** e estão no banco de dados SQLLite. Se não cadastrou dados não vai aparecer nada.

Ainda não existe controle sobre os dados cadastrados. O que será feito nas próximas etapas, junto com botões de link para a tela de cadastrar.

Inserir Botões de navegação entre as activity’s

1) Poderíamos colocar um botão na tela ‘Cadastrar’ para ir ao ‘ListarAlunos’?

2) Poderíamos colocar um botão na tela ‘ListarAlunos’ para ir ao ‘Cadastrar’?



Como poderíamos fazer isso?

Obs: Cadastrar é chamado de ‘MainActivity.java’ no nosso projeto.

Inserir Botões de navegação entre as activity's

Resposta:

Devemos inserir um botão no layout (xml) de cada tela ('MainActivity' e 'ListarAlunosActivity', ancorar ele para não ficar solto, tratar o evento 'onClick' para quando o botão for clicado levar a outra tela.

Qual o método que faz isso?

```
//método para botão irParaListar qdo clicado
public void irParaListar(View view) {
    Intent intent = new Intent(this, ListarAlunosActivity.class);
    startActivity(intent);
}
```

Inserir Botões de navegação entre as activity’s

O que é um Intent no Android?

- **Definição**

Um **Intent** em Android é um objeto que permite a comunicação entre diferentes componentes de uma aplicação, como atividades, serviços e broadcast receivers. Ele é usado para solicitar uma ação de outro componente, seja dentro da mesma aplicação ou de uma aplicação diferente.

- **Tipos de Intents**

Existem dois tipos principais de intents:

1.Explicit Intent (Intent Explícito): Especifica o componente exato que deve ser chamado pela intenção. Geralmente é usado para iniciar uma atividade ou serviço dentro da própria aplicação.

Ex:

```
Intent intent = new Intent(this, ListarAlunosActivity.class);
startActivity(intent);
```

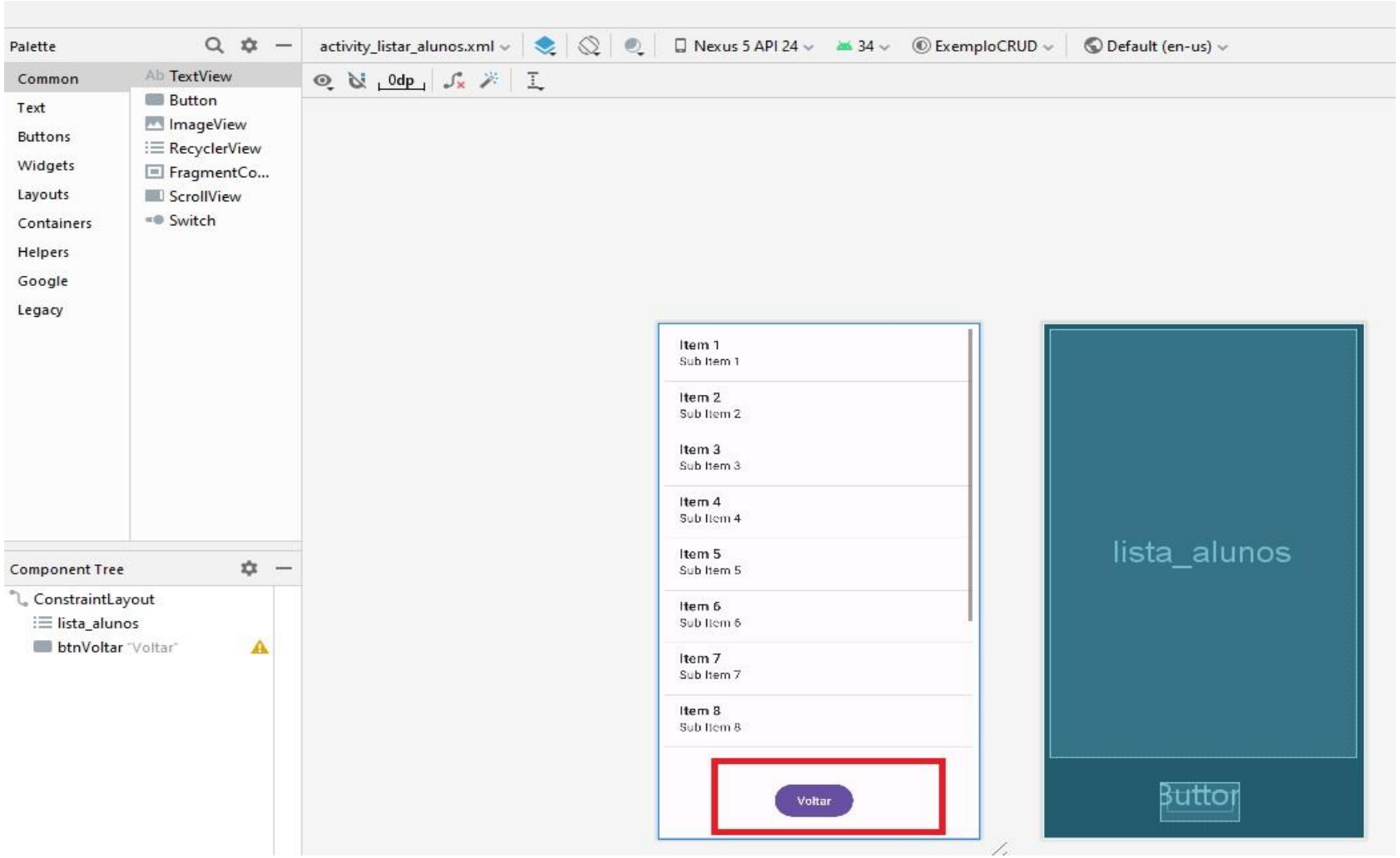
Inserir Botões de navegação entre as activity’s

1. **Implicit Intent (Intent Implícito)**: Não especifica o componente exato, mas declara uma ação geral que pode ser tratada por qualquer aplicativo capaz de executar essa ação. Geralmente é usado para interagir com outros aplicativos.

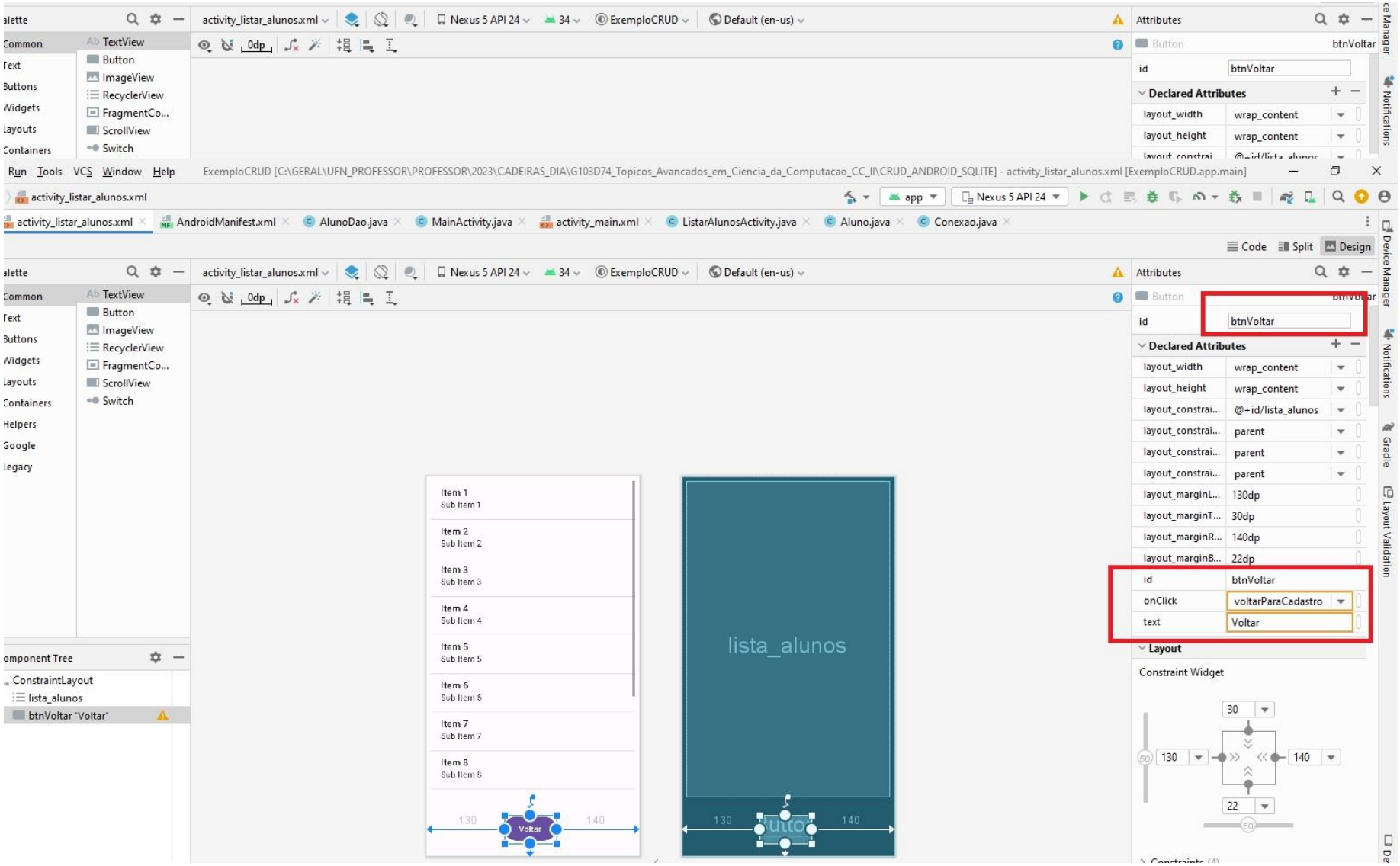
2. Ex:

```
Intent intent = new Intent(Intent.ACTION_VIEW, Uri.parse("http://www.example.com"));
startActivity(intent);
```

Inserir Botões de navegação entre as activity’s



Inserir Botões de navegação entre as activity’s



Vamos usar id=‘btnVoltar’

onClick=‘voltarParaCadastro’

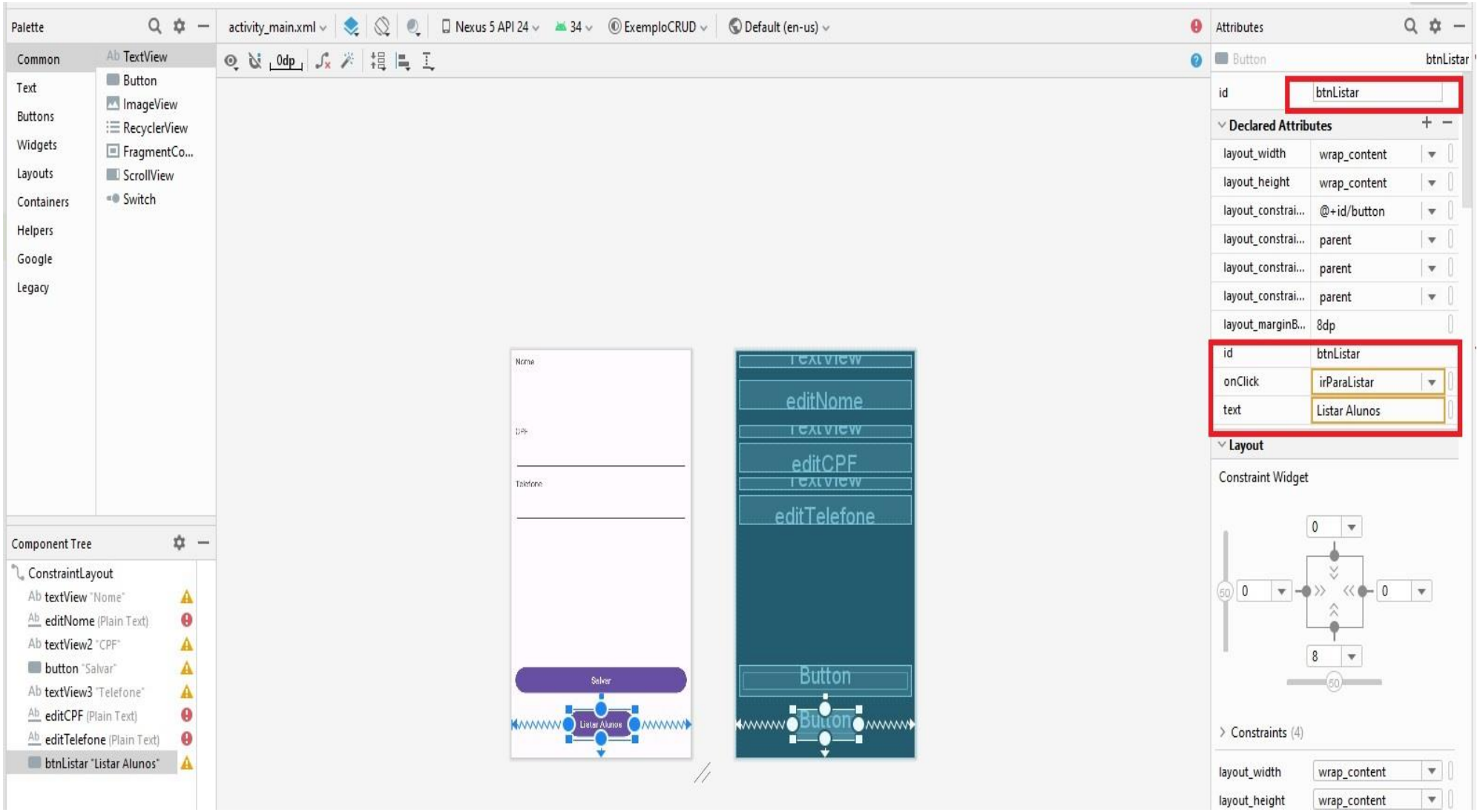
Text=‘Voltar’

Inserir Botões de navegação entre as activity's

```
7       tools:context=".ListarAlunosActivity">
8
9       <ListView
10          android:id="@+id/lista_alunos"
11          android:layout_width="0dp"
12          android:layout_height="0dp"
13          android:layout_marginStart="8dp"
14          android:layout_marginLeft="8dp"
15          android:layout_marginTop="8dp"
16          android:layout_marginEnd="8dp"
17          android:layout_marginRight="8dp"
18          android:layout_marginBottom="100dp"
19          app:layout_constraintBottom_toBottomOf="parent"
20          app:layout_constraintEnd_toEndOf="parent"
21          app:layout_constraintStart_toStartOf="parent"
22          app:layout_constraintTop_toTopOf="parent" />
23
24      <Button
25          android:id="@+id/btnVoltar"
26          android:layout_width="wrap_content"
27          android:layout_height="wrap_content"
28          android:layout_marginStart="130dp"
29          android:layout_marginLeft="130dp"
30          android:layout_marginTop="30dp"
31          android:layout_marginEnd="140dp"
32          android:layout_marginRight="140dp"
33          android:layout_marginBottom="22dp"
34          android:onClick="voltarParaCadastro"
35          android:text="Voltar"
36          app:layout_constraintBottom_toBottomOf="parent"
37          app:layout_constraintEnd_toEndOf="parent"
38          app:layout_constraintStart_toStartOf="parent"
39          app:layout_constraintTop_toBottomOf="@+id/lista_alunos" />
40  </androidx.constraintlayout.widget.ConstraintLayout>
```

Verificar se no código da activity_listar_alunos.xml
Se foi realmente alterada as ações que foram programadas

Inserir Botões de navegação entre as activity's



Fazer o mesmo processo na activity_main.xml E verificar o código como anteriormente.

Inserir Botões de navegação entre as activity's

activity_listar_alunos.xml | AndroidManifest.xml | AlunoDao.java | MainActivity.java | activity_main.xml | ListarAlunosActivity.java

```java
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d( tag: "MainActivity",  msg: "MainActivity onCreate: Activity started");


        //Vinculando os campos do layout com as variáveis do Java
        nome = findViewById(R.id.editNome);
        cpf = findViewById(R.id.editCPF);
        telefone = findViewById(R.id.editTelefone);

        dao = new AlunoDao( context: this);


    }

    //método para botão salvar qdo clicado
    2 usages
    public void salvar(View view){
        Aluno a = new Aluno();
        a.setNome(nome.getText().toString());
        a.setCpf(cpf.getText().toString());
        a.setTelefone(telefone.getText().toString());

        long id = dao.inserir(a); //inserir o aluno
        Toast.makeText( context: this, text: "Aluno inserido com id: " +id, Toast.LENGTH_SHORT).show();
    }


    //método para botão irParaLitar qdo clicado
    1 usage
    public void irParaListar(View view) {
        Intent intent = new Intent( packageContext: this, ListarAlunosActivity.class);
        startActivity(intent);
    }


}
```

Inserindo método 'irParaListar' na 'MainActivity.java'

Inserir Botões de navegação entre as activity’s

Agora faça o mesmo na
‘ListarAlunosActivity.java’

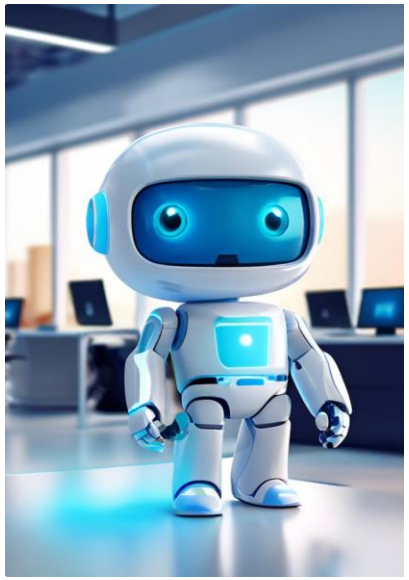
Método
public void voltarParaCadastro(View view){

Complete
}

Inserir Botões de navegação entre as activity’s

Após tudo estar pronto, veja se o arquivo AndroidManifest.xml está configurado corretamente para executar a ‘MainActivity’ como principal novamente!

Lembre-se de cadastrar algum usuário para poder visualizar no ‘ListarAlunosActivity.java’.



Inserir Botões de navegação entre as activity's

Resultado deve ser:

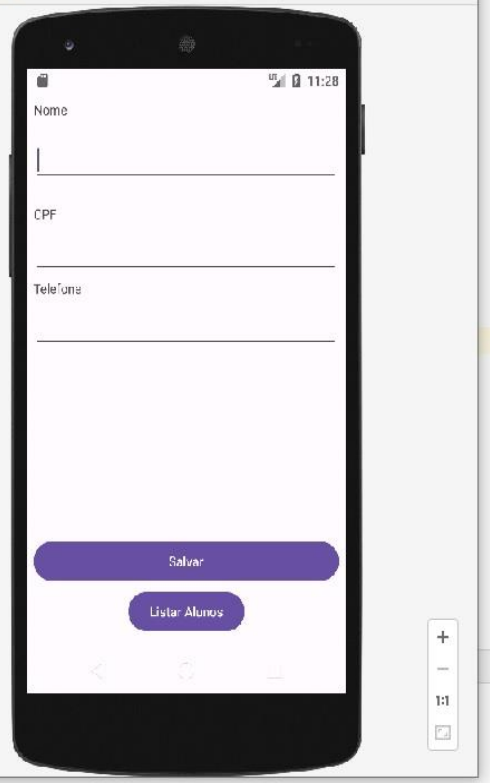
```
Android
app
manifests
AndroidManifest.xml
java
com.example.exemplocrud
Aluno
AlunoDao
Conexao
ListarAlunosActivity
MainActivity
com.example.exemplocrud (androidTest)
com.example.exemplocrud (test)
java (generated)
res
drawable
ic_launcher_background.xml
ic_launcher_foreground.xml (v24)
layout
activity_listar_alunos.xml
activity_main.xml
menu
menu_principal.xml
mipmap
navigation
values
attrs.xml
colors.xml
dimens (4)
refs.xml
strings.xml
styles.xml
themes (3)
```

activity_listar_alunos.xml | AndroidManifest.xml | AlunoDao.java | MainActivity.java | activity_main.xml | ListarAlunosActivity.java | Aluno.java | Conexao.java

```
        setContentView(R.layout.activity_main);
        Log.d( tag: "MainActivity",  msg: "MainActivity onCreate: Activity started");


        //Vinculando os campos do layout com as variáveis do Java
        nome = findViewById(R.id.editNome);
        cpf = findViewById(R.id.editCPF);
        telefone = findViewById(R.id.editTelefone);

        dao = new AlunoDao( context: this);


    }

    //método para botão salvar qdo clicado
    2 usages
    public void salvar(View view){
        Aluno a = new Aluno();
        a.setNome(nome.getText().toString());
        a.setCpf(cpf.getText().toString());
        a.setTelefone(telefone.getText().toString());

        long id = dao.inserir(a); //inserir o aluno
        Toast.makeText( context: this, text: "Aluno inserido com id: " +id, Toast.LENGTH_SHORT).show();
    }


    //método para botão irParaLitar qdo clicado
    1 usage
```

Running Devices - ExemploCRUD

Running Devices: Nexus 5 API 24




```
Run: app
2024-08-05 08:27:59: Launching app on 'Nexus 5 API 24.
$ adb shell am start -n "com.example.exemplocrud/com.example.exemplocrud.MainActivity" -a android.intent.action.MAIN -c android.intent.category.LAUNCHER

Starting: Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.example.exemplocrud/.MainActivity }

Open logcat panel for emulator Nexus 5 API 24
Connected to process 29934 on device 'Nexus_5_API_24 [emulator-5554]'.
```

Inserir Botões de navegação entre as activity’s

Resultado deve ser:

```
        setContentView(R.layout.activity_main);
        Log.d( tag: "MainActivity",  msg: "MainActivity onCreate: Activity started");


        //Vinculando os campos do layout com as variáveis do Java
        nome = findViewById(R.id.editNome);
        cpf = findViewById(R.id.editCPF);
        telefone = findViewById(R.id.editTelefone);

        dao = new AlunoDao( context: this);


    }

    //método para botão salvar qdo clicado
    2 usages
    public void salvar(View view){
        Aluno a = new Aluno();
        a.setNome(nome.getText().toString());
        a.setCpf(cpf.getText().toString());
        a.setTelefone(telefone.getText().toString());

        long id = dao.inserir(a); //inserir o aluno
        Toast.makeText( context: this, text: "Aluno inserido com id: " +id, Toast.LENGTH_SHORT).show();
    }


    //método para botão irParaLitar qdo clicado
    1 usage
```

```
Run: app
2024-08-05 08:27:59: Launching app on 'Nexus 5 API 24.
$ adb shell am start -n "com.example.exemplocrud/com.example.exemplocrud.MainActivity" -a android.intent.action.MAIN -c android.intent.category.LAUNCHER

Starting: Intent { act=android.intent.action.MAIN cat=[android.intent.category.LAUNCHER] cmp=com.example.exemplocrud/.MainActivity }

Open logcat panel for emulator Nexus 5 API 24
Connected to process 29934 on device 'Nexus_5_API_24 [emulator-5554]'.
```

Exercício de aula - Validação de Dados no Cadastrar

Após fazer a parte de navegação com botões entre uma tela e outra (intent), vamos fazer a validação dos dados digitados pelo usuário, devem ser validados:

- a) Verificar se os campos a digitar estão vazios.

- b) CPF (procurar código de validação e inserir no projeto), se for inválido mostrar uma mensagem na tela até o usuário digitar correto.

- c) Telefone (procurar código de validação e inserir no projeto), se for inválido mostrar uma mensagem na tela até o usuário digitar correto.

- Colocar os novos métodos dentro da Classe ‘AlunoDao.java’ para manter as boas práticas e organizar o novo código da ‘MainActivity.java’ ou ‘CadastrarAluno.java’ que é a nossa classe principal.

Exercício de aula - Validação de Dados no Cadastrar

Dicas: Faça as validações dos dados ao clicar no botão ‘salvar’. Os métodos chamados devem estar dentro do AlunoDao.java de preferência ou seguir as boas práticas de programação.

unoDao.java | MainActivity.java | ListarAlunosActivity.java | Aluno.java | activity_listar_alunos.xml | activity_main.xml | Conexao.java

```java
}

//----------------------------//método para botão salvar qdo clicado-----------------------------------------------------------------//
1 usage
public void salvar(View view){

    String nomeDigitado = nome.getText().toString().trim();
    String cpfDigitado = cpf.getText().toString().trim();
    String telefoneDigitado = telefone.getText().toString().trim();

    // Verifica se os campos estão vazios
    if (nomeDigitado.isEmpty() || cpfDigitado.isEmpty() || telefoneDigitado.isEmpty()) {
        Toast.makeText( context: this,  text: "Preencha todos os campos!", Toast.LENGTH_SHORT).show();
        return;
    }

    // Validação do CPF (verifica se o formato e os dígitos são válidos)
    if (!dao.validaCpf(cpfDigitado)) {
        Toast.makeText( context: this,  text: "CPF inválido. Digite novamente.", Toast.LENGTH_SHORT).show();
        return;
    }

    // Verifica se o CPF já existe no banco de dados
    if (dao.cpfExistente(cpfDigitado)) {
        Toast.makeText( context: this,  text: "CPF duplicado. Insira um CPF diferente.", Toast.LENGTH_SHORT).show();
        return;
    }

    // Validação do Telefone
    if (!dao.validaTelefone(telefoneDigitado)) {
        Toast.makeText( context: this,  text: "Telefone inválido! Use o formato correto: (XX) 9XXXX-XXXX", Toast.LENGTH_SHORT).show();
        return;
    }
```

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

O **CPF (Cadastro de Pessoa Física)** tem **11 dígitos numéricos**, sendo que os **nove primeiros são a base do número**, e os **dois últimos são dígitos verificadores** gerados a partir dos primeiros.
A **validação do CPF** segue as regras da **Receita Federal do Brasil** e funciona da seguinte forma:

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

Estrutura do CPF
O CPF tem **11 dígitos**, divididos assim:
AAA.BBB.CCC-DD
•**AAA.BBB.CCC** → São os **9 primeiros dígitos** (números-base).
•**DD** → São os **2 últimos dígitos** (dígitos verificadores).

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

Regras para CPF Válido

1. Deve conter exatamente 11 dígitos numéricos.
2. Não pode ser uma sequência repetida (00000000000, 11111111111, etc.).
3. O primeiro dígito verificador (D1) deve ser calculado corretamente.
4. O segundo dígito verificador (D2) deve ser calculado corretamente.
5. Os dígitos verificadores (D1 e D2) devem estar corretos com base nos 9 primeiros dígitos.

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

Cálculo do Primeiro Dígito Verificador (D1)

1. Pegamos os 9 primeiros números do CPF.
2. Multiplicamos cada número por um peso de 10 a 2.
3. Somamos os produtos.
4. O resto da divisão por 11 define o primeiro dígito (D1):

Se resto < 2, então D1 = 0.
Caso contrário, D1 = 11 - resto.

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

Exemplo do Cálculo de D1
CPF: 529.982.247-XX (onde XX são os dígitos verificadores)

1 Pegamos os 9 primeiros números:
5 2 9 9 8 2 2 4 7

2 Multiplicamos pelos pesos de 10 a 2:
(5×10) + (2×9) + (9×8) + (9×7) + (8×6) + (2×5) + (2×4) + (4×3) + (7×2) = 295

3 Pegamos o resto da divisão por 11:
295 ÷ 11 = 26, resto = 9

4 Aplicamos a regra:

Se resto < 2, então D1 = 0
Caso contrário, D1 = 11 - 9 = **2**

✅ Primeiro dígito verificador (D1) é 2.

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

Cálculo do Segundo Dígito Verificador (D2)

1. Pegamos os 10 primeiros números, incluindo D1.
2. Multiplicamos cada número por um peso de 11 a 2.
3. Somamos os produtos.
4. O resto da divisão por 11 define D2:

Se resto < 2, então D2 = 0.
Caso contrário, D2 = 11 - resto.

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

Exemplo do Cálculo de D2

1 Pegamos os 10 primeiros números (incluindo D1):
5 2 9 9 8 2 2 4 7 2

2 Multiplicamos pelos pesos de 11 a 2:

Copiar
Editar
(5×11) + (2×10) + (9×9) + (9×8) + (8×7) + (2×6) + (2×5) + (4×4) + (7×3) + (2×2) = 347

3 Pegamos o resto da divisão por 11:
347 ÷ 11 = 31, resto = 6

4 Aplicamos a regra:
D2 = 11 - 6 = **5**

✅ Segundo dígito verificador (D2) é 5.

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

CPF Final
Nosso CPF original era 529.982.247-XX, e calculamos os dígitos verificadores:

✅ CPF válido: 529.982.247-25.

Exercício de aula – AlunoDao.java

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

AlunoDao.java | MainActivity.java | ListarAlunosActivity.java | Aluno.java | activity_listar_alunos.xml | activity_main.xml | Cone

```java
    }



    //--------------------------------VALIDAR CPF ----------------------------------//
    /*Verifica se o CPF é uma sequência repetida ou tem tamanho incorreto.
    Calcula o primeiro dígito verificador e verifica se é válido.
    Calcula o segundo dígito verificador e verifica se é válido.
    Se ambos os dígitos estiverem corretos, o CPF é considerado válido.*/
    1 usage
    public boolean validaCpf(String CPF) {
        // Exibe a entrada recebida (usado para depuração)
        System.out.println("String de entrada do método: " + CPF);

        // Remove espaços e caracteres não numéricos (caso tenha sido digitado com . ou -)
        CPF = CPF.replaceAll( regex: "[^0-9]",  replacement: "");

        // Verifica se o CPF tem exatamente 11 dígitos
        if (CPF.length() != 11) {
            return false;
        }

        // Verifica se o CPF não é uma sequência repetida (como 00000000000, 11111111111, etc.)
        /*
        \\d Representa qualquer número de 0 a 9
        (\\d)   Captura o primeiro dígito do CPF
        \\1 Repete o mesmo dígito capturado antes
        {10}    Exige que o mesmo dígito apareça mais 10 vezes, totalizando 11 dígitos iguais */
        if (CPF.matches( regex: "(\\d)\\1{10}")) {
            return false;
        }

        char dig10, dig11; // Variáveis para armazenar os dígitos verificadores
        int soma, num, peso, resto;
```

Exercício de aula – AlunoDao.java

Continuação.

Dicas: Dicas para implementação dos métodos no AlunoDao.java

- validaCpf() é necessário seguir as regras da receita federal.

AlunoDao.java | MainActivity.java | ListarAlunosActivity.java | Aluno.java | activity_listar_alunos.xml | activity_main.xml | Conexao.java

```java
        }

        char dig10, dig11; // Variáveis para armazenar os dígitos verificadores
        int soma, num, peso, resto;

        try {
            // Cálculo do Primeiro Dígito Verificador (D1)
            soma = 0;
            peso = 10; // O primeiro peso começa em 10 e vai diminuindo até 2

            for (int i = 0; i < 9; i++) {
                num = CPF.charAt(i) - '0'; // Converte o caractere numérico para inteiro
                soma += (num * peso); // Multiplica pelo peso correspondente e soma
                peso--; // Diminui o peso
            }

            resto = soma % 11;
            dig10 = (resto < 2) ? '0' : (char) ((11 - resto) + '0'); // Se resto < 2, D1 = 0, senão D1 = 11 - resto

            // Cálculo do Segundo Dígito Verificador (D2)
            soma = 0;
            peso = 11; // Agora os pesos começam em 11 e vão até 2

            for (int i = 0; i < 10; i++) { // Inclui o primeiro dígito verificador já calculado
                num = CPF.charAt(i) - '0'; // Converte o caractere numérico para inteiro
                soma += (num * peso); // Multiplica pelo peso correspondente e soma
                peso--; // Diminui o peso
            }

            resto = soma % 11;
            dig11 = (resto < 2) ? '0' : (char) ((11 - resto) + '0'); // Se resto < 2, D2 = 0, senão D2 = 11 - resto

            // Comparação dos Dígitos Verificadores
            return (dig10 == (char) (CPF.charAt(9))) && (dig11 == (char) (CPF.charAt(10))); // Verifica se os dígitos calculados são iguais aos do CPF informado
            //se for verdadeiro retorna true

        } catch (Exception e) { // Captura qualquer erro inesperado
            return false;
        }
    }
```

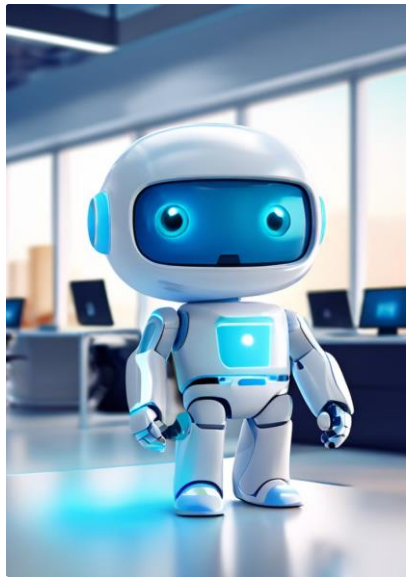
Gerar o APK do app (instalador)

Para gerar o instalador (APK) do seu aplicativo Android e compartilhá-lo com outras pessoas, você precisa seguir alguns passos no Android Studio. Aqui estão os passos detalhados para gerar um **APK (modo depuração):**

Passos para gerar o APK no Android Studio:

- Abra o seu projeto no Android Studio
- Certifique-se de que seu projeto está aberto e compilando corretamente.
- Configure as opções de build:
- Vá para Build > Build Bundle(s) / APK(s) > Build APK(s).
- O Android Studio começará a compilar seu projeto e gerar o APK.

Gerar o APK do app (instalador)



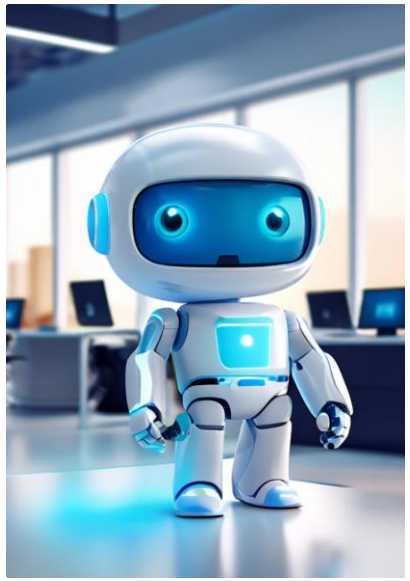
Localize o APK:

- Após a conclusão do build, o Android Studio exibe uma notificação no canto inferior direito.
- Clique em locate (localizar) para abrir o diretório onde o APK foi gerado.
- O APK geralmente é salvo na pasta **app/build/outputs/apk/debug** para builds de debug ou **app/build/outputs/apk/release** para builds de release.

Gerar o APK do app (instalador)

Gerar APK para Release

Para gerar um APK que você pode distribuir publicamente, é melhor criar uma versão de release assinada. Aqui estão os passos para isso:



Configure uma Keystore:

- Vá para Build > Generate Signed Bundle / APK....
- Escolha APK e clique em Next.
- Selecione ou crie uma nova keystore. Uma keystore é um arquivo que contém informações de segurança e é necessário para assinar seu APK.

- Preencha os campos necessários e clique em Next.

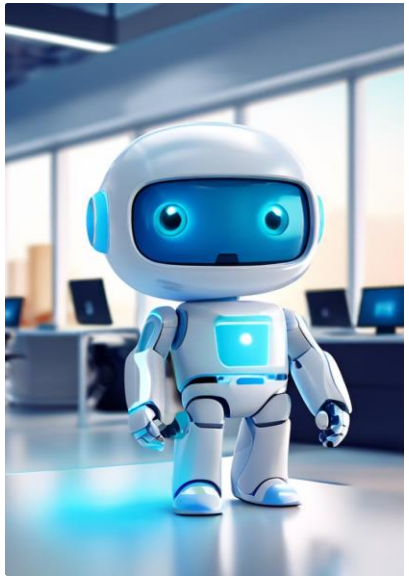
Gerar o APK do app (instalador)

Configure a assinatura do APK:

- Selecione o tipo de build (geralmente release).
- Clique em Finish.
- Build o APK:
- O Android Studio irá gerar o APK assinado.
- Após a conclusão, o Android Studio exibirá uma notificação. Clique em locate para abrir o diretório onde o APK assinado foi salvo.

Exercícios:

- Após concluir todas as etapas da aula, faremos exercícios.
- Agora temos uma lista de exercícios para revisar os conhecimentos, esta deverá ser entregue na atividade da aula de hoje.
- Ela está disponível no Minha UFN na aula de hoje.



Referências:

LECHETA, Ricardo R. Google Android: aprenda a criar aplicações para dispositivos móveis com o android SDK. São
Paulo: Novatec, 2009.
TALUKDER, Asoke; YAVAGAL, Roopa. Mobile computing. New Delhi - India: McGraw-Hill, 2007.
SILVA, D. Desenvolvimento para dispositivos móveis. Pearson, 2017.
ANDROID. Android Developers. Disponível em https://developer.android.com. Acesso em agosto de 20018. 2018.
MIKKONEN, T. Programming mobile devices: an introduction for practitioners. Chichester,
England: Wiley, 2007.
ROGERS, Rick et al. Desenvolvimento de aplicações Android. O'Reilly: Novatec, 2009.
SILVA, D. Arquitetura para computação móvel. Pearson, 2018.
YAVAGAL, Roopa R.; TALUKDER, Asoke K. Mobile computing: technology, applications, and service creation. New
York, NY: McGraw-Hill, 2007.

Obrigado pela atenção!!

Contato:
Email: **andre.flores@ufn.edu.br**